

CPU9444_A1T3_yann

[如何保证学术诚信](#)

[Colab环境配置 & 原理](#)

[Colab优势](#)

[链接到Drive \(GPU\)](#)

[代码调试/运行](#)

[Japanese Character Recognition](#)

[Background](#)

[Codes](#)

[Imports](#)

[Train](#)

[Test](#)

[Main](#)

[Model](#)

[Perceptron](#)

[MLP](#)

[CNN](#)

[Result](#)

[Parameters](#)

[Accuracy](#)

[Confusion Matrix](#)

[Non-Linear Separability](#)

[6 Hidden Nodes](#)

[Learned Weights](#)

[4 Hidden Nodes - Step function](#)

[4 Hidden Nodes - Sigmoid function](#)

[Part 3 RNN. LSTM & FSM](#)

[Q1](#)

[Q2](#)

[Q3](#)

[Q4](#)

[Q5](#)

[Recurrent Neural Network](#)

[Time Dependency](#)

[Vanishing Gradient](#)

[Memory of RNN](#)

[LSTM](#)

[Gates](#)

[Additive Gradient](#)

如何保证学术诚信

授之以鱼不如授之以渔，我会给出一个Pipeline，将搜索算法用于求解游戏的流程是什么样的。找对关键问题比方法重要。我们的作业班**strictly follow University Academic Policy**，**不涉及对作业答案提供，相关问题由于政策限制不予回答，敬请谅解**。需要提前和同学们确认这一点，这几年悉尼有很多的private tutor因为涉及到直接提供solution，整个班的人都受到影响，挂了一批。避免Academic misconduct是每个人要注意的事情▲ **Echo学姐严禁触碰一切红线**，我们教学课程的出发点是为了同学们的整体利益，帮助大家进步。判断问题是否我能够回答的界限在于，能不能公开讨论？适不适合问学校老师？任何debug相关的问题学校禁止回答的。并且学校不允许关于学生拥有作业的PDF，

所以我们课上的PDF我不会上传。更多细节查看UNSW academic policy：
<https://www.student.unsw.edu.au/plagiarism/integrity>

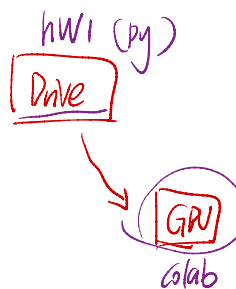
In addition, it is important that students understand that it is not permissible to buy essay/writing services from third parties as the use of such services constitutes plagiarism because it involves using the words or ideas of others and passing them off as your own. Nor is it permissible to sell copies of lecture or tutorial notes as students do not own the rights to this intellectual property.

Contract cheating, or 'ghostwriting', is when a student engages another person to complete work for them, and then submits the work as their own, regardless if money was paid or not.

Colab环境配置 & 原理

Colab优势

- 不需要预装环境（TensorFlow对系统要求很高，不好装）
- 免费的GPU（MacBook本地没有GPU）
- 方便Team work



链接到Drive (GPU)

The screenshot shows a Google Colab notebook interface. On the left, the file explorer shows a directory structure: `gdrive` -> `MyDrive` -> `AI-yann` -> `9444` -> `hw1`. The code editor on the right contains the following code blocks:

```
[1] 1 from google.colab import drive
    2
    3 drive.mount('/content/gdrive')
Mounted at /content/gdrive

1 %cd '/content/gdrive/MyDrive/AI-yann/9444/hw1'
/content/gdrive/MyDrive/AI-yann/9444/hw1

1 !ls
anb2n.py  check_main.py  data  kuzu.py  plot  __pycache__  seq_plot.py
check.csv  check.py  kuzu_main.py  net  pt1_data  seq_models.py  seq_train.py

[4] 1 !nvidia-smi
Thu Jun 20 06:52:58 2024
+-----+
| NVIDIA-SMI 535.104.05              Driver Version: 535.104.05   CUDA Version: 12.2     |
| GPU   Name           Persistence-M| Bus-Id        Disp.A    Volatile Uncorr. ECC  |
| Fan  Temp  Perf    Pwr:Usage/Cap|      Memory-Usage  GPU-Util  Compute M. |
|-----+-----+-----+-----+
|  0  Tesla T4             Off      | 00000000:00:04:0 Off    |          0%      Default MIG M. |
|    36C    P8             9W / 70W |  0MiB / 15360MiB      0%      N/A  |
+-----+-----+-----+-----+
```

代码调试/运行

Model	MNIST [16]	Kuzushiji-MNIST	Kuzushiji-49
4-Nearest Neighbour Baseline	97.14%	91.56%	86.01%
Keras Simple CNN Benchmark [4]	99.06%	95.12%	89.25%
PreActResNet-18 [11]	99.56%	97.82%	96.64%
PreActResNet-18 + Input Mixup [26]	99.54%	98.41%	97.04%
PreActResNet-18 + Manifold Mixup [22]	99.54%	98.83%	97.33%

同时你也可以去看看原文的GitHub，看看他的模型是怎么写的：

Kuzushiji-MNIST

License: CC BY-SA 4.0

[Read the paper](#) to learn more about Kuzushiji, the datasets and our motivations for making them!

News and Updates

IMPORTANT: If you downloaded the KMNIST or K49 dataset before 5 February 2019, please re-download the dataset and run your code again. We fixed minor image processing bugs and released an updated version, we find that the updated version gives slightly better performance. Thanks to #1 and #5 for bringing this to our attention.

The Dataset

Kuzushiji-MNIST is a drop-in replacement for the MNIST dataset (28x28 grayscale, 70,000 images), provided in the original MNIST format as well as a NumPy format. Since MNIST restricts us to 10 classes, we chose one character to represent each of the 10 rows of Hiragana when creating Kuzushiji-MNIST.

Kuzushiji-49, as the name suggests, has 49 classes (28x28 grayscale, 270,912 images), is a much larger, but imbalanced dataset containing 48 Hiragana characters and one Hiragana iteration mark.

Kuzushiji-Kanji is an imbalanced dataset with a total of 3,832 Kanji characters (64x64 grayscale, 140,424 images), ranging from 1,766 examples to only a single example per class.

The 10 classes of Kuzushiji-MNIST, with the first column showing each character's modern hiragana counterpart.

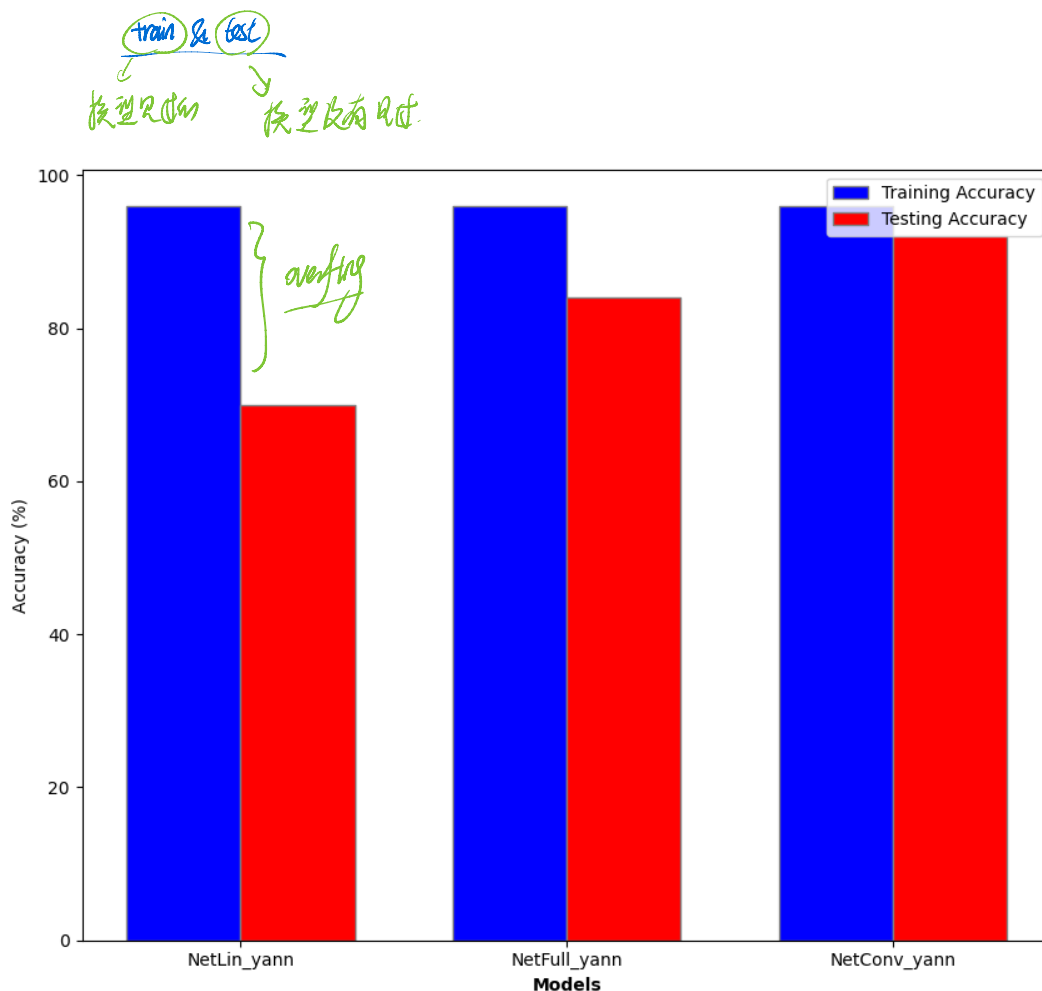
<https://github.com/rois-codh/kmnist>

原文的代码已经在这里了，所以只要照着改改就行，可以得到和原文章一样的准确性，只不过这里文章用的是Keras而不是PyTorch，Keras是更加简单一点的：

kmnist / benchmarks / kuzushiji_mnist_cnn.py

Code Blame 73 lines (61 loc) · 2.35 KB

```
3 from __future__ import print_function
4 import keras
5 from keras.models import Sequential
6 from keras.layers import Dense, Dropout, Flatten
7 from keras.layers import Conv2D, MaxPooling2D
8 from keras import backend as K
9 import numpy as np
10
11 batch_size = 128
12 num_classes = 10
13 epochs = 12
14
15 # input image dimensions
16 img_rows, img_cols = 28, 28
17
18 def load(f):
19     return np.load(f)['arr_0']
20
21 # Load the data
22 x_train = load('kmnist-train-imgs.npz')
23 x_test = load('kmnist-test-imgs.npz')
24 y_train = load('kmnist-train-labels.npz')
25 y_test = load('kmnist-test-labels.npz')
26
27 if K.image_data_format() == 'channels_first':
28     x_train = x_train.reshape(x_train.shape[0], 1, img_rows, img_cols)
29     x_test = x_test.reshape(x_test.shape[0], 1, img_rows, img_cols)
30     input_shape = (1, img_rows, img_cols)
31 else:
32     x_train = x_train.reshape(x_train.shape[0], img_rows, img_cols, 1)
33     x_test = x_test.reshape(x_test.shape[0], img_rows, img_cols, 1)
34     input_shape = (img_rows, img_cols, 1)
35
36 x_train = x_train.astype('float32')
37 x_test = x_test.astype('float32')
38 x_train /= 255
39 x_test /= 255
40 print('{} train samples, {} test samples'.format(len(x_train), len(x_test)))
41
42 # Convert class vectors to binary class matrices
43 y_train = keras.utils.to_categorical(y_train, num_classes)
44 y_test = keras.utils.to_categorical(y_test, num_classes)
45
46 model = Sequential()
47 model.add(Conv2D(32, kernel_size=(3, 3),
48                 activation='relu',
49                 input_shape=input_shape))
50 model.add(Conv2D(64, (3, 3), activation='relu'))
51 model.add(MaxPooling2D(pool_size=(2, 2)))
52 model.add(Dropout(0.25))
53 model.add(Flatten())
54 model.add(Dense(128, activation='relu'))
55 model.add(Dropout(0.5))
56 model.add(Dense(num_classes, activation='softmax'))
57
58 model.compile(loss=keras.losses.categorical_crossentropy,
59              optimizer=keras.optimizers.Adadelta(),
60              metrics=['accuracy'])
61
62 model.fit(x_train, y_train,
63         batch_size=batch_size,
64         epochs=epochs,
65         verbose=1,
66         validation_data=(x_test, y_test))
67
68 train_score = model.evaluate(x_train, y_train, verbose=0)
69 test_score = model.evaluate(x_test, y_test, verbose=0)
70 print('Train loss:', train_score[0])
71 print('Train accuracy:', train_score[1])
72 print('Test loss:', test_score[0])
73 print('Test accuracy:', test_score[1])
```

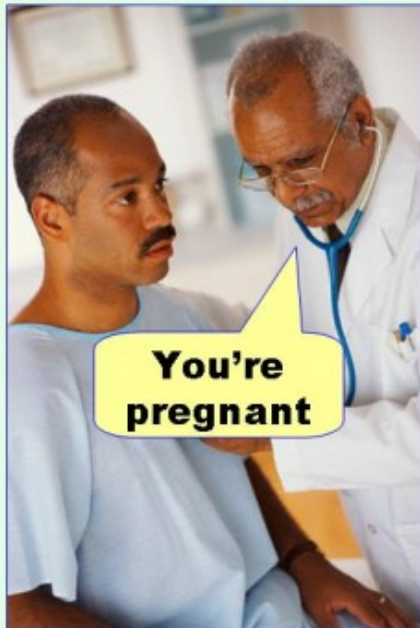



Confusion Matrix

混淆矩阵是一种用于评估分类模型性能的表格表示方法。它显示了模型对每个类别的预测结果和实际标签之间的关系。对于二分类问题，混淆矩阵通常包含四个基本组成部分：

- **True Positive (TP):** 模型正确地预测了正类。
- **True Negative (TN):** 模型正确地预测了负类。
- **False Positive (FP):** 模型错误地将负类预测为正类 (Type I Error)。
- **False Negative (FN):** 模型错误地将正类预测为负类 (Type II Error)。

Type I error
(false positive)



Type II error
(false negative)



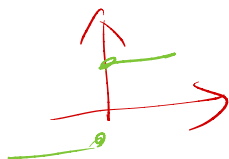
对于二分类问题，混淆矩阵通常如下所示：

		预测类别	
		Positive	Negative
实际类别	Positive	TP	FN
	Negative	FP	TN

核心代码和最终结果：

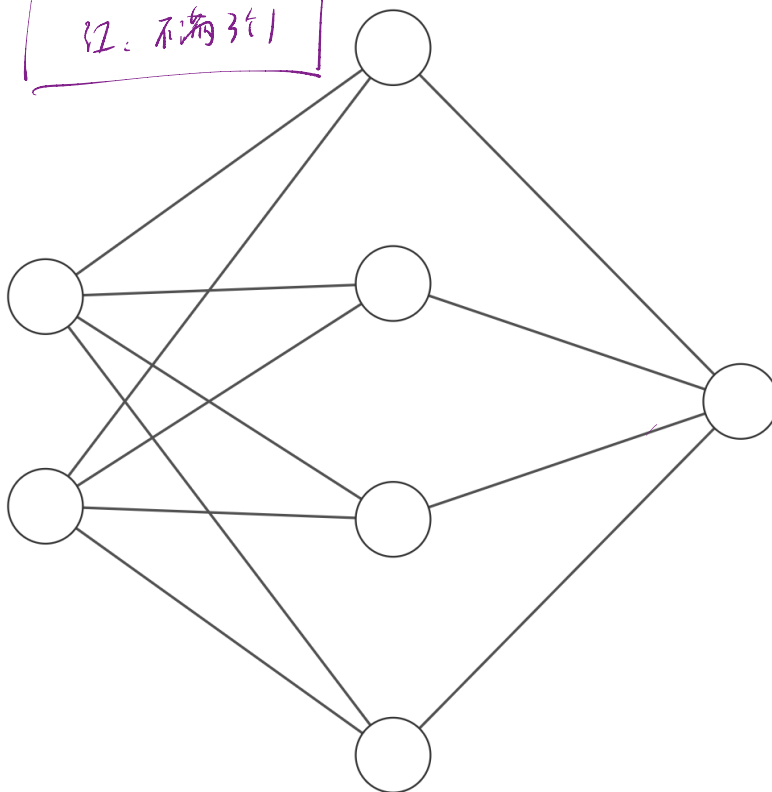
```
import seaborn as sns

sns.heatmap(conf_matrix)
```



第二层的输出：
蓝：至少3个
红：不满3个

$$+|x| + |x| + 0x = \underline{3} - 2 = \underline{1}$$



$$= \underline{0}$$

Input Layer $\in \mathbb{R}^2$

Hidden Layer $\in \mathbb{R}^4$

Output Layer $\in \mathbb{R}^1$

结合我们的数据输入，其实是这样的：

蓝 $\rightarrow 1$ 红 $\rightarrow 0$ 0%

x_1	x_2	$f(X)$	y
-1	-1	1, 1, 1, 0	1
-1	0	1, 0, 0, 1	0
-1	1	1, 0, 1, 1	1
0	-1		0
0	0		1
0	1		0
1	-1		1
1	0		0
1	1		0

最终结果：

Part 3 RNN. LSTM & FSM

这个5个questions，这五个问题对于往年的同学来说是最难的作业。Q1是训练RNN分析的anbn输出结果，输出图像画一下结果。Q2是画出FSM，Q3是分析RNN的结果。Q4用LSTM跑更复杂的语言anbncn，Q5是分析结果。

Q1，2和3可以放在一起做，简单来说，Q1设置的目的是为了让你理解题目的要求，理解作业代码训练RNN输出的东西是什么，Q2的目的是利用Q1的结果来画出Finite State Machine。所以简单来说，这两问的任务是想要让RNN来模拟Finite State Machine。FSM这个概念lecture上面是没有的，我把知识点补充进来，Q3是解释RNN的运行原理。

Q1

RNN → 规则. → $n=2$ aa bbbb
*a → a? 0.9
→ b? 0.1*
vector [0.9, 0.1] → Probability
第一个语言是anbn²，希望RNN学习规律，比如先n个a然后再2n个b。比如第一个symbol是
AAABBBBBBAABBBBBABBABBAABBBBBBBBA

```
symbol= AAABBBBBBAABBBBBABBABBAABBBBBBBBA
hidden activations and output probabilities:
```

```
A [ 0. -0.] [ 0.5 0.5]
A [ 0. -0.] [ 0.5 0.5]
A [ 0. -0.] [ 0.5 0.5]
B [ 0. -0.] [ 0.5 0.5]
B [ 0. -0.] [ 0.5 0.5]
B [ 0. -0.] [ 0.5 0.5]
B [ 0. -0.] [ 0.5 0.5]
B [ 0. -0.] [ 0.5 0.5]
B [ 0. -0.] [ 0.5 0.5]
A [ 0. -0.] [ 0.5 0.5]
A [ 0. -0.] [ 0.5 0.5]
B [ 0. -0.] [ 0.5 0.5]
B [ 0. -0.] [ 0.5 0.5]
B [ 0. -0.] [ 0.5 0.5]
B [ 0. -0.] [ 0.5 0.5]
A [ 0. -0.] [ 0.5 0.5]
B [ 0. -0.] [ 0.5 0.5]
B [ 0. -0.] [ 0.5 0.5]
A [ 0. -0.] [ 0.5 0.5]
A [ 0. -0.] [ 0.5 0.5]
A [ 0. -0.] [ 0.5 0.5]
A [ 0. -0.] [ 0.5 0.5]
B [ 0. -0.] [ 0.5 0.5]
B [ 0. -0.] [ 0.5 0.5]
B [ 0. -0.] [ 0.5 0.5]
B [ 0. -0.] [ 0.5 0.5]
B [ 0. -0.] [ 0.5 0.5]
B [ 0. -0.] [ 0.5 0.5]
B [ 0. -0.] [ 0.5 0.5]
B [ 0. -0.] [ 0.5 0.5]
```

→ RNN 还没有经过训练。

epoch: 0
loss: 0.1967

训练完之后的结果是:

symbol= AAABBBBB ABB AAABBBBB ABB AAAABBBBBBBB AA

hidden activations and output probabilities:

A [0.61 -0.8] [0.81 0.19]

A [0.21 -0.64] [0.66 0.34]

A [-0.34 -0.42] [0.4 0.6]

B [-0.71 1.] [0. 1.] 2

B [-0.27 0.63] [0. 1.] 3

B [0.16 0.72] [0. 1.] 4

B [0.77 -0.14] [0.03 0.97] 5

B [0.87 0.77] [0. 1.] 6

B [0.98 -0.92] [0.87 0.13] 0.5

A [0.64 -0.81] [0.82 0.18]

B [0.46 1.] [0. 1.]

B [0.95 -0.89] [0.85 0.15]

A [0.62 -0.82] [0.83 0.17]

A [0.21 -0.63] [0.64 0.36]

A [-0.32 -0.46] [0.46 0.54]

B [-0.72 1.] [0. 1.]

B [-0.28 0.63] [0. 1.]

B [0.15 0.72] [0. 1.]

B [0.76 -0.13] [0.03 0.97]

B [0.87 0.76] [0. 1.]

B [0.98 -0.91] [0.87 0.13]

A [0.64 -0.82] [0.82 0.18]

B [0.46 1.] [0. 1.]

B [0.95 -0.89] [0.85 0.15]

A [0.62 -0.82] [0.83 0.17] ✓

A [0.21 -0.63] [0.64 0.36]

A [-0.32 -0.46] [0.46 0.54]

A [-0.81 0.04] [0.04 0.96]

B [-0.87 1.] [0. 1.]

B [-0.5 0.77] [0. 1.]

B [-0.12 0.74] [0. 1.]

B [0.5 0.31] [0. 1.]

B [0.85 0.33] [0. 1.]

B [0.95 -0.35] [0.11 0.89]

B [0.9 0.85] [0. 1.]

B [0.98 -0.95] [0.9 0.1]

epoch: 9

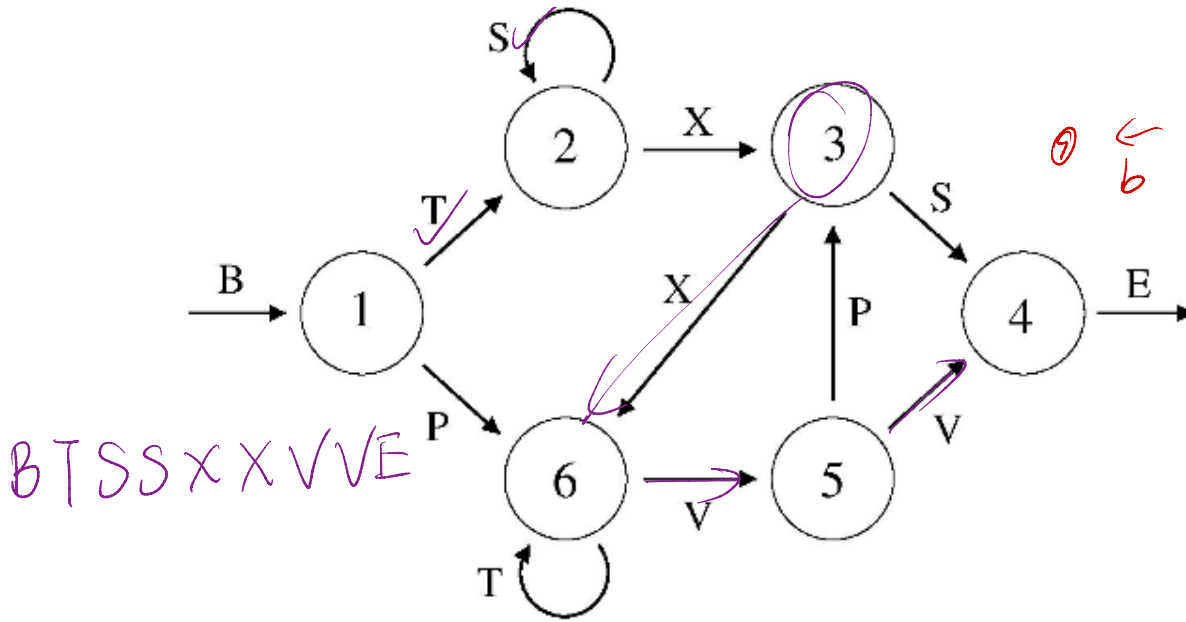
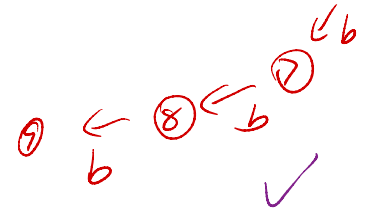
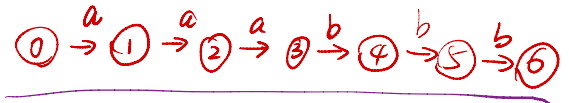
loss: 0.0321

输出结果:

A. A. A. B. B. B. B. B. B.

$n=3$

$n=4$



BTSSXXVVE

有限状态机 (Finite State Machine, FSM) 是一种抽象的数学模型，用于描述系统的行为和状态转换。它是一种有限个状态和在这些状态之间转移的规则的数学模型，被定义为：

1. **状态 (States)**：系统可能处于的每一个状态都是有限的、确定的，并且是离散的。状态可以表示系统在不同时间上的内部条件或者模式。
2. **转移 (Transitions)**：定义了系统在不同状态之间如何转换的规则。转移可以由外部输入触发，也可以是系统内部条件的结果。
3. **初始状态 (Initial State)**：系统在开始时的初始状态。
4. **终止状态 (Final State)**：一个或多个特定的状态，系统在到达这些状态时可能结束或者停止。

FSM可以被用来用于词法分析和语法分析，FSM是自动机的一种，图灵机是更高级的自动机，可以进行编译计算机程序。有个词叫做图灵可计算性，代表的含义就是图灵机可以解决的算法。这部分内容是Model of computation，计算机原理的内容。

Q3

解释RNN的工作原理

[1 mark] Briefly explain how the network accomplishes the $a^n b^{2n}$ task. Specifically, you should describe how the hidden unit activations change as the string is processed, and how it is able to correctly predict all B's after the first B, as well as the initial A following the last B in the sequence.

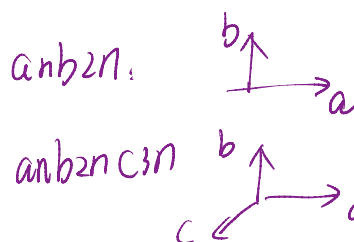
$\hookrightarrow$ RNN $\rightarrow a^n b^{2n}$.

n 确定了 $\rightarrow$ 序列才确定。

① 识别 B $\rightarrow n \rightarrow$ 后面 b 正确。

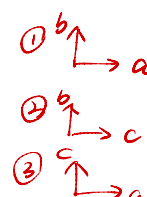
Q4

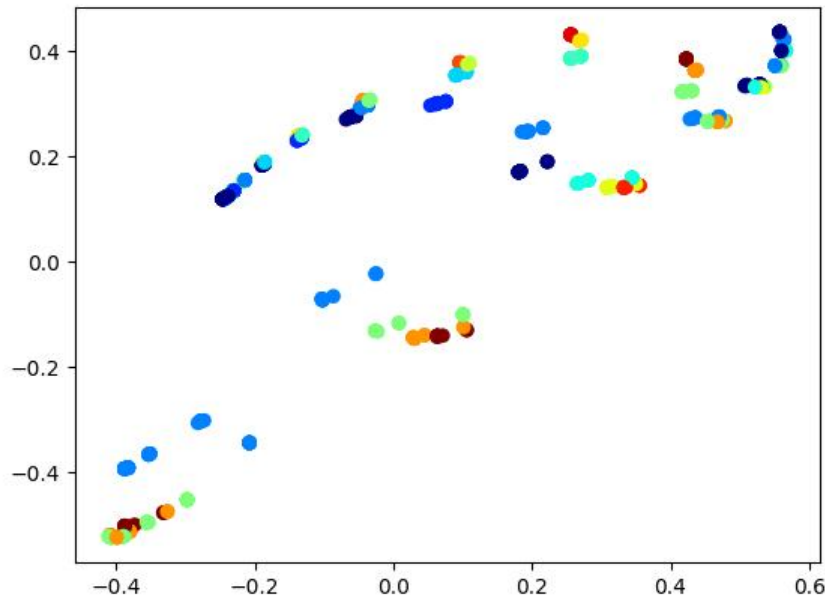
这里有更加复杂的语言， anb^2nc^3n ，这里有3个变量，所以有3个轴。问的问题和第一问是一样的，但是这里需要考虑到3轴的信息：



3D $\rightarrow$ 3张图。

② n 确定了 $\rightarrow$ B 确定 $\rightarrow$ A





Q5

这里涉及到LSTM内存的机制。

→ 没有内存
 RNN: 没有办法达到长时间序列。
 → 处理长时间序列存在困难。
 LSTM: 有内存 → 能提长时间序列。
 T: a a a a b b b b b b b b c c c c . -
 8b 12
 n=100

Recurrent Neural Network

循环神经网络（RNN）是一类能够处理序列数据的神经网络模型，其关键特征是能够记住先前输入的信息，并使用这些信息来影响当前的输出。这使得RNN在自然语言处理等需要处理时间序列数据的任务中非常有效。

Time Dependency

在处理自然语言时，我们的输入是一串序列，其中的每个单词叫做一个token。token之间的位置关系被视为时序关系，因为输入的顺序会影响其含义。例如：

"I saw a saw"
 "我看到了一把锯子"

在这个例子中，虽然"saw"在句子中出现了两次，但由于其在句子中的位置不同，它们的含义完全不同。第一个"saw"在"I"的后面，作为动词使用，意思是"看见"。第二个"saw"跟在量词"a"后面，作为名词使用，意思是"锯子"。RNN通过其循环结构能够处理这种时序依赖关系。

Vanishing Gradient

RNN的核心特征是其循环结构，这种结构允许信息从一个时间步骤传递到下一个时间步骤。具体来说，RNN的每个单元不仅接收当前时间步骤的输入，还接收前一个时间步骤的隐藏状态。这使得RNN能够"记住"前面的信息，并将其用于当前的计算。

□ Formally

□ x_t is the input

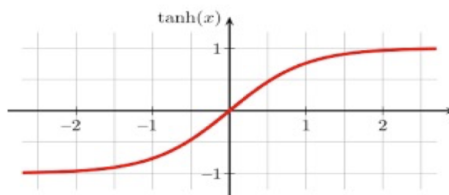
□ h_t is the hidden state

□ y_t is the output

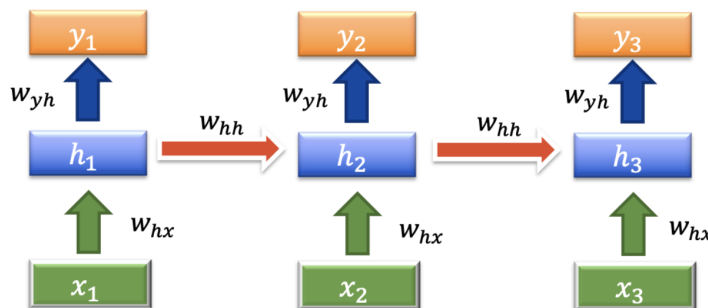
□ $h_0 = \vec{0}$

□ $h_t = \tanh(w_{hh}h_{t-1} + w_{hx}x_t + b_h)$

□ $y_t = w_{yh}h_t$



The University of Sydney



Page 13

USYD COMP5329 Lecture slides - Week8

在训练RNN时，我们需要对时间步进行展开，这意味着RNN在每个时间步都有两个输入：当前时间步的输入 x_t 和前一时间步的隐藏状态 h_{t-1} 。具体的计算如下：

$$h_t = \tanh(W_{hh}h_{t-1} + W_{hx}x_t + b_h)$$

这种递归关系可以展开成前面的多个时间步，比如：

$$h_t = \tanh(W_{hh} \cdot \tanh(W_{hh} \cdot \tanh(\dots \tanh(W_{hh}h_0 + W_{hx}x_1 + b_h) + W_{hx}x_2 + b_h) + \dots) + W_{hx}x_t + b_h)$$

在反向传播中，我们需要计算损失函数相对于网络参数的梯度。对于RNN，这涉及到对多个时间步展开的求导。具体来说，损失函数 L 对隐藏状态 h_t 的梯度计算涉及链式法则：

$$\frac{\partial L}{\partial W_{hh}} = \sum_{t=1}^T \frac{\partial L}{\partial h_t} \frac{\partial h_t}{\partial W_{hh}}$$

由于 h_t 是通过 h_{t-1} 递归计算得到的，链式法则将引入多次相乘的梯度项：

$$\frac{\partial h_t}{\partial W_{hh}} = \frac{\partial h_t}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial h_{t-2}} \dots \frac{\partial h_1}{\partial W_{hh}}$$

这种链式的梯度计算导致了梯度的指数级变化。当权重矩阵 W_{hh} 的特征值大于1时，梯度会呈指数级增长，导致梯度爆炸；当特征值小于1时，梯度会呈指数级衰减，导致梯度消失。具体来说：

- **梯度爆炸 (Gradient Explosion)**：如果权重矩阵的特征值大于1，梯度会随着时间步的增加呈指数级增长。这会导致权重更新幅度过大，使得模型参数不稳定，甚至发生数值溢出。
- **梯度消失 (Gradient Vanishing)**：如果权重矩阵的特征值小于1，梯度会随着时间步的增加呈指数级衰减。这会导致梯度逐渐变得非常小，使得模型几乎无法更新参数，训练过程停止。

□ Vanishing gradient problem

$$\square h_t = \tanh(w_{hh}h_{t-1} + w_{hx}x_t + b_h)$$

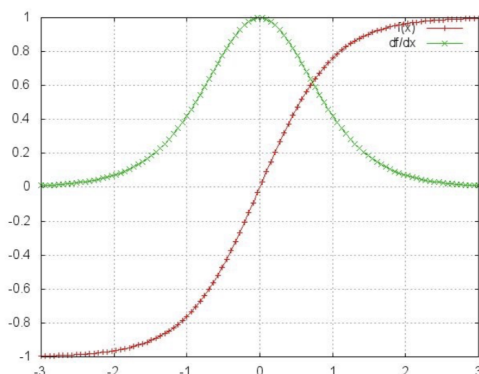
$$\square y_t = w_{yh}h_t$$

□ For every step, its loss is E_n

$$\frac{\partial E_3}{\partial w_{hh}} = \frac{\partial E_3}{\partial y_3} \frac{\partial y_3}{\partial h_3} \frac{\partial h_3}{\partial w_{hh}}$$

$$(\tanh)'w_{hh}(\tanh)'w_{hh}(\tanh)'w_{hh} \dots$$

$$\begin{aligned} 0.9^{1000} &\approx 0 \\ 1.01^{1000} &\approx 21000 \end{aligned}$$



$$(\tanh)' \leq 1$$

If $0 < w_{hh} < 1$, **Vanishing Gradients**

$$(\tanh)'w_{hh}(\tanh)'w_{hh}(\tanh)'w_{hh} \dots \rightarrow 0$$

If w_{hh} is larger, **Exploding Gradients**

$$(\tanh)'w_{hh}(\tanh)'w_{hh}(\tanh)'w_{hh} \dots \rightarrow \infty$$

The University of Sydney

Page 15

USYD COMP5329 Lecture slides - Week8

在训练长序列时，RNN可能会遇到梯度消失或梯度爆炸问题。这使得模型难以学习到长距离的依赖关系。RNN的计算是顺序的，这使得并行化处理困难，训练时间较长。RNN难以捕捉到长序列中的长期依赖关系，虽然LSTM和GRU等变体在一定程度上缓解了这个问题的挑战。

Memory of RNN

尽管RNN没有明确的内存模块，但它通过其循环连接能够记住一些信息，并在后续的时间步骤中使用这些信息。例如，当我们说“故人西辞黄鹤楼”，RNN可能会预测下一句是“烟花三月下扬州”。这种记忆是通过模型的权重和结构涌现出来的。

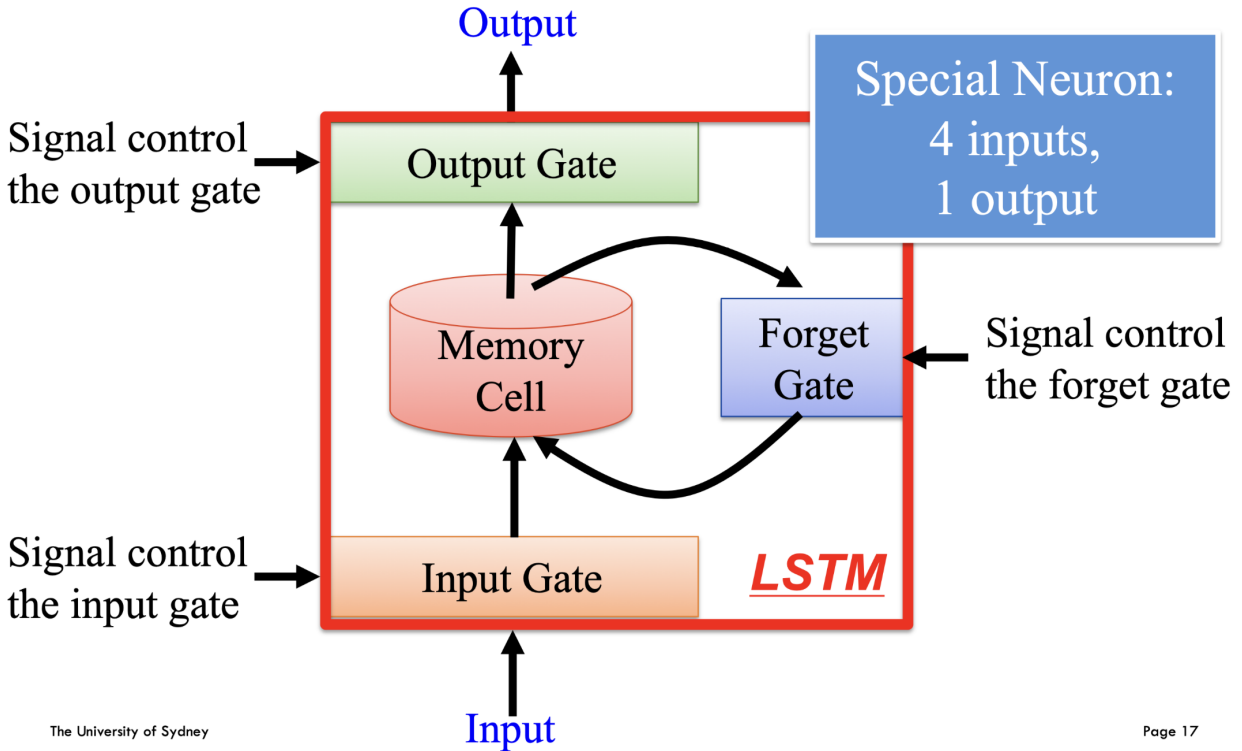
“涌现现象”是复杂系统理论中的一个概念，指从简单的规则中自然产生复杂的行为。在RNN的上下文中，尽管每个单独的神经元功能相对简单，整个网络却能通过这些简单的交互产生复杂的行为模式，如语言中的语法结构和长期依赖关系的学习。这种从简单到复杂的行为是涌现的一种形式。

LSTM

LSTM (Long Short-Term Memory) 网络是一种能够有效解决RNN梯度消失问题的变体。通过引入门控机制，LSTM在处理长序列时表现出色。以下是对LSTM的详细介绍，特别是门控特性和如何解决RNN梯度消失问题。

Gates

LSTM包含三个主要的门：输入门、遗忘门和输出门。这些门通过控制信号（通常使用sigmoid函数）来调节信息流，从而实现对信息的选择性记忆和遗忘。



USYD COMP5329 Lecture slides - Week8

- **输入门 (Input Gate)**：决定哪些新信息需要存储到单元状态。

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

- **遗忘门 (Forget Gate)**：决定哪些信息需要从单元状态中删除。

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

- **输出门 (Output Gate)**：基于当前的单元状态和输入，决定最终的输出。

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

- **单元状态更新**：单元状态 (Cell State) 存储跨时间步的信息，并在每个时间步进行更新。

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

$$C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t$$

$$h_t = o_t \cdot \tanh(C_t)$$

□ Candidate cell state

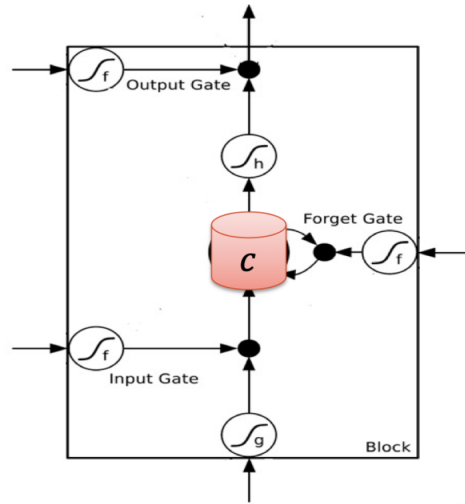
$$\bullet \tilde{c}_t = \tanh(w_{hh}h_{t-1} + w_{hx}x_t + b_h)$$

□ Update cell state

$$\bullet c_t = f_t * c_{t-1} + i_t * \tilde{c}_t$$

□ Output state

$$\bullet h_t = o_t * \tanh(c_t)$$



The University of Sydney

28

USYD COMP5329 Lecture slides - Week8

Additive Gradient

LSTM通过加性梯度而非乘性梯度解决了RNN的梯度消失问题。具体来说，LSTM的单元状态 C_t 直接通过遗忘门和输入门进行加性更新，而不是像RNN那样逐步通过非线性激活函数传播，这使得梯度不会在长序列中指数级衰减。

• RNN的梯度传播：

$$\frac{\partial L}{\partial h_t} = \frac{\partial L}{\partial h_{t+1}} \cdot \frac{\partial h_{t+1}}{\partial h_t} + \frac{\partial L}{\partial y_t} \cdot \frac{\partial y_t}{\partial h_t}$$

由于激活函数的存在， $\frac{\partial h_{t+1}}{\partial h_t}$ 会导致梯度逐步衰减或爆炸。

• LSTM的梯度传播：

$$\frac{\partial L}{\partial C_t} = \frac{\partial L}{\partial C_{t+1}} \cdot \frac{\partial C_{t+1}}{\partial C_t} + \frac{\partial L}{\partial h_t} \cdot \frac{\partial h_t}{\partial C_t}$$

由于 C_t 的更新是加性的（ $C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t$ ），梯度不会在长序列中指数级衰减。

LSTM通过引入输入门、遗忘门和输出门，门控机制允许LSTM选择性地记忆和遗忘信息，有效地解决了RNN的梯度消失问题。其加性梯度更新机制使得LSTM能够处理长序列数据，捕捉长期依赖关系。然而，LSTM也因其复杂性和计算成本较高而存在一定的局限性。通过合理的设计和优化，LSTM在自然语言处理等领域表现出色。